

Cyber Security Beaconing Analysis for Command-and- Control C2 Detection.

Supervisor : Scott Orr (IU OmniSOC)
Students : Mohammed Ambusaidi and Nolan Knies

Introduction

C2 beaconing is a common technique used in cyberattacks, where infected systems communicate with external servers in periodic patterns. These behaviors are difficult to detect as they often resemble normal traffic.

This project uses **unsupervised machine learning** on network flow data to detect beaconing activity and generate anomaly scores, helping distinguish malicious communication from normal behavior.

Objectives

- Detect **C2 beaconing behavior** using anomaly detection
- Distinguish **normal vs. malicious traffic** using anomaly scores
- Reduce **false positives** and improve ranking reliability
- Provide **stable and interpretable results** for analysts

Data

We use the **CTU-13 dataset**, which contains network traffic with both normal activity and attack scenarios. Each record represents a **network flow** with statistical features describing communication behavior.

- **~56,000 flows** (~5% attacks)

- **Features**: timing, packet size, duration, and traffic ratios
- **Preprocessing**: cleaning, transformation, and feature engineering

We created **30+ additional features** to capture patterns related to beaconing behavior, such as timing regularity and traffic asymmetry.

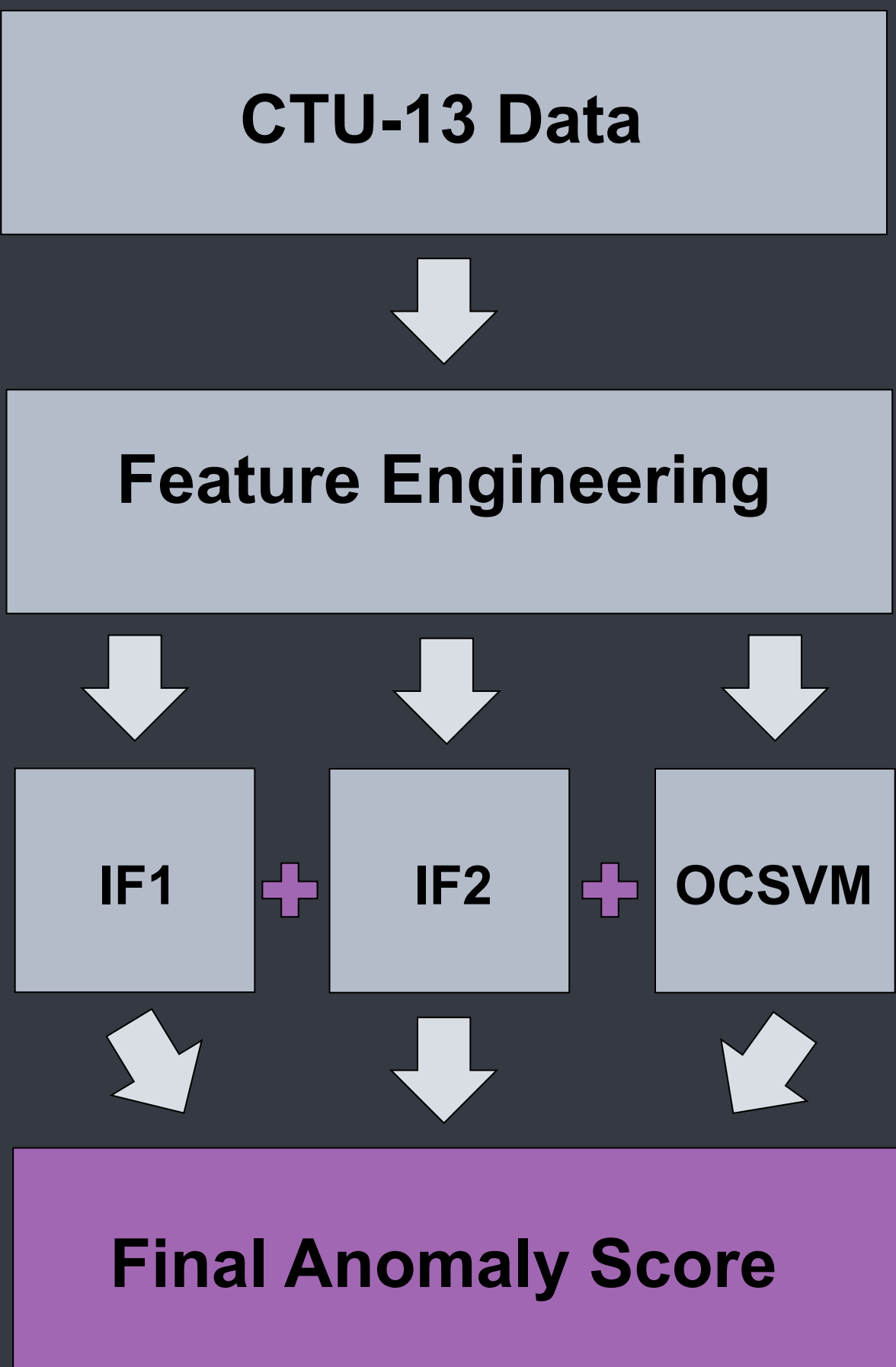
Method

- **Baseline**: Isolation Forest applied to CTU-13 network flow data to detect anomalous communication patterns
- Produces anomaly scores (normal vs suspicious traffic)

Improvements

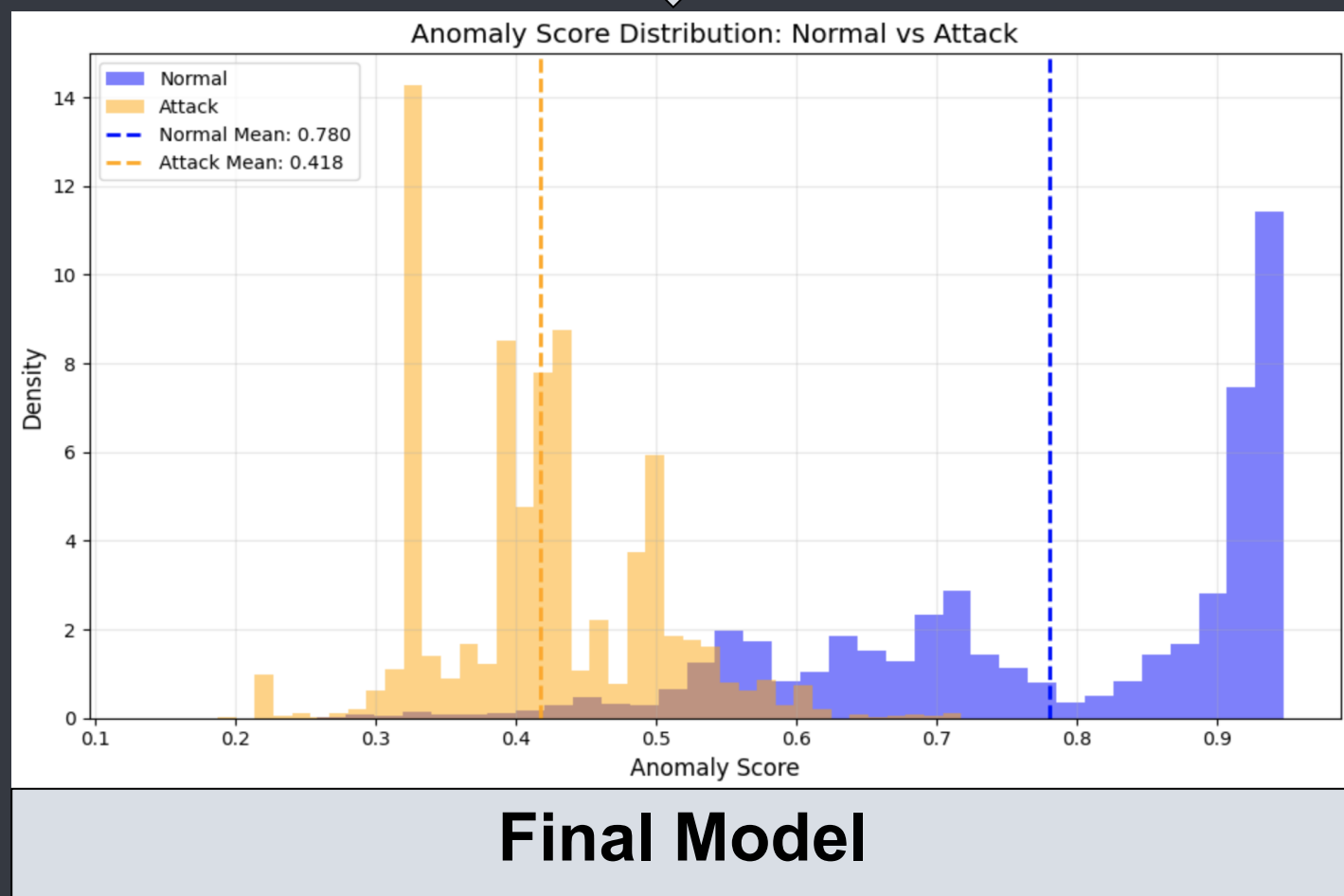
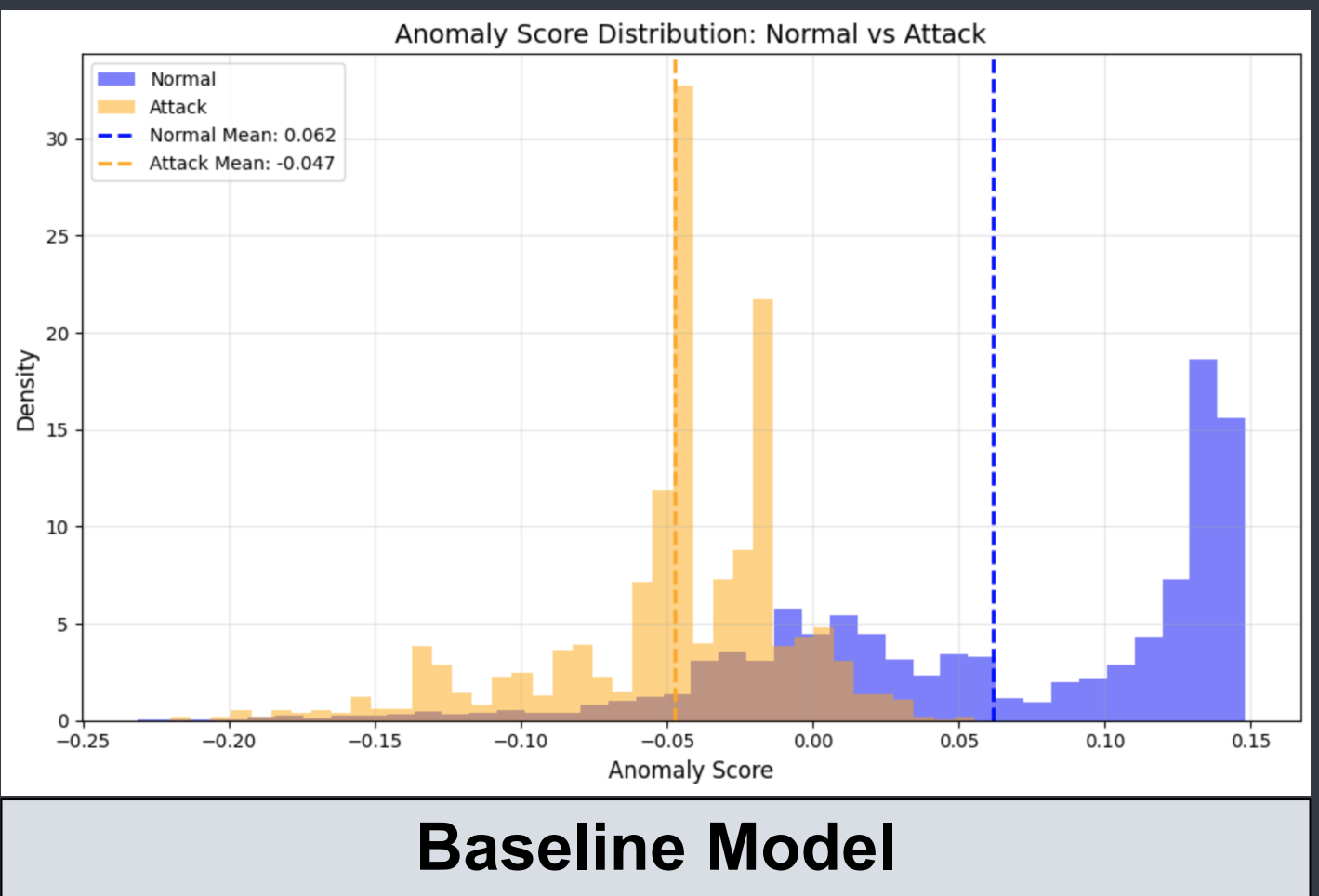
- **Improvement 1**: Added One-Class SVM → improved separation and classification performance
- **Improvement 2**: Feature engineering (30+ features) + dual Isolation Forest models (IF1, IF2)
- **Improvement 3 (Final Model)**:
 1. Ensemble of **IF1 + IF2 + OCSVM**
 2. Optimized weights (~0.09 / 0.59 / 0.32)
 3. Yeo-Johnson transformation for robust scaling
 4. Selected **IF2-heavy model for stability and consistency**

Model Overview



Results

Model Stage	Separation	Key Insight
Baseline IF	Low (~0.1)	High overlap
IF1 + IF2 + OCSVM	~0.4	Improved separation
Final Ensemble	~0.37 (stable)	Best overall performance



Final Metrics

- **AUC-ROC: 0.973**
- **KS Statistic: 0.863**
- **Separation: ~0.37**
- **Stability: consistent across 20 runs**
- **Top 10% anomalies → ~90% attacks captured**

Conclusion

- Final ensemble model achieves strong separation between normal and attack traffic
- Provides **reliable and stable anomaly scores** for real-world use
- Effectively prioritizes suspicious activity (**~90% attacks in top 10%**)
- Suitable as a support tool for cybersecurity analysts

Key Takeaways

- The model separates normal and attack traffic much better than before
- Combining multiple models improved the overall performance
- Feature engineering helped reduce errors and overlap
- The model is good at finding important attacks early (top 10%)

References

- CTU-13 Botnet Dataset (Czech Technical University)
- Pedregosa et al., *Scikit-learn: Machine Learning in Python*
- Python Libraries: NumPy, Pandas, Matplotlib
- IU OmniSOC (Project Context)

Future Work

- Test the model on real IU OmniSOC data
- Integrate the model with **Elastic SIEM** for **real-time detection and alerting**
- Improve attack precision and reduce false positives
- Explore additional methods (e.g., clustering or deep learning)